

```

# IoT Smart Home Dashboard - Application-Specific Deadlock Analysis

## 🕒 **WHERE DEADLOCKS OCCUR IN YOUR APPLICATION**

---

## **Scenario 1: Timer Service + Manual Device Control** 🚨 **HIGHEST RISK**

### **Exact Location in Your Code**:
- **File**: `SmartHomeService.java:174-253` (changeGadgetStatus method)
- **File**: `TimerService.getInstance()` background threads
- **File**: `SmartHomeDashboard.java:1384-1465` (scheduleDeviceTimer method)

### **Real Deadlock Scenario**:
```java
// THREAD 1: User manually controlling device
(SmartHomeDashboard.java:975-997)
private static void changeGadgetStatus() {
 List<Gadget> gadgets = smartHomeService.viewGadgets();
 // User selects device to control
 String gadgetType = selectedGadget.getType();
 String roomName = selectedGadget.getRoomName();

 // This calls SmartHomeService.changeSpecificGadgetStatus()
 smartHomeService.changeSpecificGadgetStatus(gadgetType, roomName);
}

// Inside SmartHomeService.changeSpecificGadgetStatus() (Lines 254-337)
Customer currentUser = sessionManager.getCurrentUser();
synchronized(currentUser) { // LOCK 1: Customer object
 Gadget targetGadget = currentUser.findGadget(gadgetType, roomName);
 synchronized(targetGadget) { // LOCK 2: Gadget object
 targetGadget.toggleStatus();
 customerService.updateCustomer(currentUser);
 }
}

// THREAD 2: TimerService background execution
// When timer triggers device automation
synchronized(gadget) { // LOCK 1: Gadget object (DIFFERENT ORDER!)
 gadget.toggleStatus();
 synchronized(customer) { // LOCK 2: Customer object - DEADLOCK!
 customerService.updateCustomer(customer);
 }
}
}
```

### **How We Overcome This**:
```java
// SOLUTION: Lock Ordering Manager
public class SmartHomeService {
 private static final Object LOCK_ORDER = new Object();
}
```

```

```

        // Modified changeSpecificGadgetStatus method
        public boolean changeSpecificGadgetStatus(String gadgetType, String
roomName) {
            Customer currentUser = sessionManager.getCurrentUser();

            // Always lock Customer first, then Gadget
            return executeWithConsistentLocking(currentUser, gadgetType,
roomName, () -> {
                Gadget targetGadget = currentUser.findGadget(gadgetType,
roomName);
                if (targetGadget != null) {
                    String previousStatus = targetGadget.getStatus();
                    targetGadget.toggleStatus();

                    boolean updated =
customerService.updateCustomer(currentUser);
                    if (updated) {
                        sessionManager.updateCurrentUser(currentUser);
                        return true;
                    }
                }
                return false;
            });
        }

```

```

        private boolean executeWithConsistentLocking(Customer customer,
String deviceType,
String roomName,
Supplier<Boolean> operation) {
            synchronized(customer) { // Always Customer FIRST
                Gadget device = customer.findGadget(deviceType, roomName);
                if (device != null) {
                    synchronized(device) { // Always Gadget SECOND
                        return operation.get();
                    }
                }
            }
            return false;
        }
    }
}
...

```

Scenario 2: Group Management + Device Access 🚨 **CRITICAL RISK**

Exact Location in Your Code:

- **File**: `SmartHomeService.java:1293-1329` (grantDevicePermission)
- **File**: `SmartHomeService.java:353-407` (addPersonToGroup)
- **File**: `Customer.java:232-249` (getAccessibleGroupDevices)

Real Deadlock Scenario:

```java

```
// THREAD 1: Group admin granting device permission
(SmartHomeService.java:1293-1329)
public boolean grantDevicePermission(String memberEmail, String
deviceType, String roomName) {
 Customer currentUser = sessionManager.getCurrentUser(); // Admin user

 synchronized(currentUser) { // LOCK 1: Admin Customer
 if (!currentUser.isGroupAdmin()) return false;

 // Find the device in admin's collection
 Gadget device = currentUser.findGadget(deviceType, roomName);
 synchronized(device) { // LOCK 2: Device
 Customer memberUser =
customerService.findCustomerByEmail(memberEmail);
 synchronized(memberUser) { // LOCK 3: Member Customer
 return currentUser.grantDevicePermission(memberEmail,
deviceType, roomName,
currentUser.getEmail());
 }
 }
 }
}
```

```
// THREAD 2: Group member accessing shared device
public boolean changeSpecificGadgetStatus(String gadgetType, String
roomName) {
 Customer currentUser = sessionManager.getCurrentUser(); // Member
user

 synchronized(currentUser) { // LOCK 1: Member Customer
 // Need to access admin's device through group permissions
 Customer adminUser = findDeviceOwner(gadgetType, roomName);
 synchronized(adminUser) { // LOCK 2: Admin Customer (DIFFERENT
ORDER!)
 Gadget device = adminUser.findGadget(gadgetType, roomName);
 synchronized(device) { // LOCK 3: Device - POTENTIAL
DEADLOCK!
 device.toggleStatus();
 return customerService.updateCustomer(adminUser);
 }
 }
 }
}
...
}
```

### \*\*How We Overcome This\*\*:

```
```java
// SOLUTION: Hierarchical Lock Ordering + Timeout
public class GroupDeviceManager {
    private static final int TIMEOUT_SECONDS = 5;

    public boolean grantDevicePermissionSafe(String memberEmail, String
deviceType, String roomName) {
```

```

        Customer currentUser = sessionManager.getCurrentUser();
        Customer memberUser =
customerService.findCustomerByEmail(memberEmail);

        // Sort users by email to ensure consistent ordering
        Customer firstLock =
currentUser.getEmail().compareTo(memberUser.getEmail()) < 0 ?
            currentUser : memberUser;
        Customer secondLock =
currentUser.getEmail().compareTo(memberUser.getEmail()) < 0 ?
            memberUser : currentUser;

        ReentrantLock firstUserLock = getUserLock(firstLock);
        ReentrantLock secondUserLock = getUserLock(secondLock);

        try {
            if (firstUserLock.tryLock(TIMEOUT_SECONDS, TimeUnit.SECONDS))
{
                try {
                    if (secondUserLock.tryLock(TIMEOUT_SECONDS,
TimeUnit.SECONDS)) {
                        try {
                            // Safe operations with consistent ordering
                            Gadget device =
currentUser.findGadget(deviceType, roomName);
                            return
currentUser.grantDevicePermission(memberEmail, deviceType,
roomName, currentUser.getEmail());
                        } finally {
                            secondUserLock.unlock();
                        }
                    } else {
                        System.out.println("[WARNING] Could not acquire
member lock - preventing deadlock");
                        return false;
                    }
                } finally {
                    firstUserLock.unlock();
                }
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return false;
        }
        return false;
    }
}
...

```

Scenario 3: Session Management + Database Operations 🚨 **HIGH RISK**

```

### **Exact Location in Your Code**:
- **File**: `SessionManager.java` (singleton instance)
- **File**: `CustomerService.authenticateCustomer()` method
- **File**: `SmartHomeService.java:1224-1245` (updateUserFullName)

### **Real Deadlock Scenario**:
```java
// THREAD 1: User updating profile (SmartHomeService.java:1224-1245)
public boolean updateUserFullName(String newName) {
 if (!sessionManager.isLoggedIn()) return false;

 synchronized(SessionManager.getInstance()) { // LOCK 1: Session
Manager
 Customer currentUser = sessionManager.getCurrentUser();
 synchronized(currentUser) { // LOCK 2: Customer object
 currentUser.setFullName(newName);
 // Database update while holding session lock
 boolean success =
customerService.updateCustomer(currentUser); // May acquire DB lock
 if (success) {
 sessionManager.updateCurrentUser(currentUser);
 }
 return success;
 }
 }
}

// THREAD 2: Another user trying to log in
public Customer authenticateCustomer(String email, String password) {
 // Database operation might acquire DB connection lock first
 synchronized(databaseConnection) { // LOCK 1: Database connection
 Customer user = findCustomerByEmail(email);
 if (user != null && verifyPassword(password, user.getPassword()))
 {
 // Now trying to update session
 synchronized(SessionManager.getInstance()) { // LOCK 2:
Session Manager - DEADLOCK!
 sessionManager.login(user);
 return user;
 }
 }
 }
 return null;
}
```

### **How We Overcome This**:
```java
// SOLUTION: Database Transactions + Service Layer Separation
@Service
public class DeadlockSafeCustomerService {

 @Transactional(isolation = Isolation.READ_COMMITTED, timeout = 10)

```

```

 public Customer authenticateCustomer(String email, String password) {
 // Database transaction handles locking automatically
 Customer customer = customerRepository.findByEmail(email);

 if (customer != null && verifyPassword(password,
customer.getPassword())) {
 // Update user state in database first
 customer.setLastLoginTime(LocalDateTime.now());
 customer.resetFailedAttempts();
 customer = customerRepository.save(customer);

 // Session update happens outside of database transaction
 CompletableFuture.runAsync(() -> {
 sessionManager.login(customer);
 });

 return customer;
 }
 return null;
 }

 @Transactional(isolation = Isolation.READ_COMMITTED, timeout = 5)
 public boolean updateUserProfile(Customer user, String newName) {
 user.setFullName(newName);
 Customer updatedUser = customerRepository.save(user);

 // Session update in separate thread to avoid deadlock
 CompletableFuture.runAsync(() -> {
 sessionManager.updateCurrentUser(updatedUser);
 });

 return true;
 }
}
...

```

---

## \*\*Scenario 4: Alert Service + Device State Changes\*\* 🚨 \*\*MEDIUM RISK\*\*

### \*\*Exact Location in Your Code\*\*:

- \*\*File\*\*: `AlertService.java:217-233` (alert checking loops)
- \*\*File\*\*: `SmartHomeService.java:1521-1532` (forceAlertCheck)

### \*\*Real Deadlock Scenario\*\*:

```

```java
// THREAD 1: User changing device state triggers energy alert check
public boolean changeGadgetStatus(String gadgetType) {
    Customer currentUser = sessionManager.getCurrentUser();
    synchronized(currentUser) { // LOCK 1: Customer
        Gadget device = currentUser.findGadget(gadgetType, roomName);
        synchronized(device) { // LOCK 2: Device
            device.toggleStatus();
        }
    }
}

```

```

        // Energy change might trigger alert check
        synchronized(AlertService.getInstance()) { // LOCK 3: Alert
Service
    alertService.checkEnergyUsageAlerts(currentUser.getEmail(), currentUser);
        }
    }
}

```

```

// THREAD 2: Background alert monitoring
public void checkTimeBasedAlerts(String userEmail, LocalDateTime now) {
    synchronized(AlertService.getInstance()) { // LOCK 1: Alert Service
        List<Alert> userAlerts = getUserAlerts(userEmail);
        Customer customer =
customerService.findCustomerByEmail(userEmail);

```

```

        synchronized(customer) { // LOCK 2: Customer (DIFFERENT ORDER!)
            for (Gadget device : customer.getGadgets()) {
                synchronized(device) { // LOCK 3: Device - POTENTIAL
DEADLOCK!
                    checkDeviceAlerts(device, now);
                }
            }
        }
    }
}
...

```

How We Overcome This:

```

```java
// SOLUTION: Event-Driven Alert System
public class EventDrivenAlertService {
 private final BlockingQueue<AlertEvent> alertQueue = new
LinkedBlockingQueue<>();
 private final ExecutorService alertProcessor =
Executors.newSingleThreadExecutor();

 public EventDrivenAlertService() {
 // Start alert processor thread
 alertProcessor.submit(this::processAlerts);
 }

 // Non-blocking alert trigger
 public void triggerAlertCheck(String userEmail, String deviceId,
AlertType type) {
 AlertEvent event = new AlertEvent(userEmail, deviceId, type,
System.currentTimeMillis());
 alertQueue.offer(event); // Non-blocking, no deadlock possible
 }

 // Single-threaded alert processing (deadlock-free)
 private void processAlerts() {

```

```

 while (!Thread.currentThread().isInterrupted()) {
 try {
 AlertEvent event = alertQueue.take();
 processAlertEvent(event); // Safe - single thread
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 break;
 }
 }

 private void processAlertEvent(AlertEvent event) {
 // Safe processing - no concurrent access issues
 Customer customer =
customerService.findCustomerByEmail(event.getUserEmail());
 Gadget device = customer.findGadget(event.getDeviceId());

 // Check alerts without locking conflicts
 checkDeviceAlerts(device, customer);
 }
 }

// Usage in SmartHomeService
public boolean changeGadgetStatus(String gadgetType) {
 // ... device control logic ...

 // Non-blocking alert trigger instead of synchronous check
 alertService.triggerAlertCheck(currentUser.getEmail(),
device.getId(), AlertType.ENERGY_USAGE);
 return true;
}
...

Scenario 5: Smart Scenes + Manual Device Control 🚨 **MEDIUM
RISK**

Exact Location in Your Code:
- **File**: `SmartHomeService.java:778-791` (executeSmartScene)
- **File**: `SmartScenesService.executeScene()` method

Real Deadlock Scenario:
```java
// THREAD 1: User executing MOVIE scene (SmartHomeDashboard.java:3091-3114)
private static void executeSmartScene() {
    String sceneName = sceneNames.get(choice - 1);
    boolean success = smartHomeService.executeSmartScene(sceneName);
}

// Inside SmartScenesService.executeScene()
public SceneExecutionResult executeScene(String sceneName, Customer
customer, CustomerService customerService) {

```



```

        synchronized(this) { // LOCK 1: SmartScenesService singleton
            List<SceneAction> actions = getSceneActions(sceneName);

            // Scene tries to control multiple devices in sequence
            for (SceneAction action : actions) {
                Gadget device1 = customer.findGadget("TV", "Living Room");
                synchronized(device1) { // LOCK 2: TV device
                    device1.setStatus("ON");

                    Gadget device2 = customer.findGadget("LIGHT", "Living
Room");
                    synchronized(device2) { // LOCK 3: Light device (Lock
order: TV -> Light)
                        device2.setStatus("OFF");
                    }
                }
            }
        }
    }
}

```

```

// THREAD 2: User manually controlling light while scene is running
public boolean changeSpecificGadgetStatus(String gadgetType, String
roomName) {
    Customer currentUser = sessionManager.getCurrentUser();

    if ("LIGHT".equals(gadgetType)) {
        Gadget lightDevice = currentUser.findGadget("LIGHT", "Living
Room");
        synchronized(lightDevice) { // LOCK 1: Light device
            lightDevice.toggleStatus();

            // Scene might still be running, needs TV device
            if (sceneInProgress) {
                Gadget tvDevice = currentUser.findGadget("TV", "Living
Room");
                synchronized(tvDevice) { // LOCK 2: TV device (Lock
order: Light -> TV - DEADLOCK!)
                    // Some interaction between devices
                }
            }
        }
    }
}
...

```

How We Overcome This:

```

```java
// SOLUTION: Device Coordination Manager
public class DeviceCoordinationManager {
 private final Map<String, ReentrantLock> deviceLocks = new
ConcurrentHashMap<>();
 private final Map<String, Integer> deviceLockOrder = new HashMap<>();

 public DeviceCoordinationManager() {

```

```

 // Assign consistent ordering to device types
 deviceLockOrder.put("TV", 1);
 deviceLockOrder.put("LIGHT", 2);
 deviceLockOrder.put("AC", 3);
 deviceLockOrder.put("SPEAKER", 4);
 // ... other device types
 }

 public boolean executeSceneSafely(String sceneName, List<SceneAction>
actions, Customer customer) {
 // Sort actions by device lock order to prevent deadlock
 actions.sort((a1, a2) -> {
 int order1 = deviceLockOrder.getOrDefault(a1.getDeviceType(),
999);
 int order2 = deviceLockOrder.getOrDefault(a2.getDeviceType(),
999);
 return Integer.compare(order1, order2);
 });

 List<ReentrantLock> acquiredLocks = new ArrayList<>();

 try {
 // Acquire all device locks in order
 for (SceneAction action : actions) {
 String deviceKey = action.getDeviceType() + "_" +
action.getRoomName();
 ReentrantLock deviceLock =
deviceLocks.computeIfAbsent(deviceKey, k -> new ReentrantLock());

 if (deviceLock.tryLock(5, TimeUnit.SECONDS)) {
 acquiredLocks.add(deviceLock);
 } else {
 System.out.println("[WARNING] Could not acquire lock
for " + deviceKey + " - scene partially executed");
 return false;
 }
 }

 // Execute scene actions safely
 for (SceneAction action : actions) {
 Gadget device =
customer.findGadget(action.getDeviceType(), action.getRoomName());
 if (device != null) {
 device.setStatus(action.getAction());
 }
 }

 return true;
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 return false;
 } finally {
 // Release locks in reverse order

```

